

OpsGate

What it is, why it exists, and what it means for your project

Version 7.1 | August 2026

This document explains what OpsGate is, why it exists, and what changes once it's connected to a project — in plain language, for anyone who manages a project, makes decisions about scope and risk, or simply wants to understand what it does. A companion technical document covers architecture and implementation detail for engineers who need it.

1. The short version

OpsGate is a safety and consistency layer that sits between an AI coding assistant and a real software project. When an AI assistant (such as Claude or a Replit Agent) is about to make a change to your project, OpsGate checks the request against a set of rules before anything happens — what it's allowed to touch, what it's forbidden from touching, and whether a human needs to weigh in first.

Think of it as a second set of eyes that never gets tired, never skips a step, and applies the exact same standard every single time — regardless of which AI assistant is doing the work, which day it is, or how routine the task looks.

In one sentence

OpsGate makes sure an AI assistant working on your project stays inside agreed boundaries, and stops to ask a person before it does anything that genuinely requires a judgment call.

2. Why this exists

AI coding assistants are very capable, but left unsupervised they have a few real, well-known failure modes: they can drift outside the task they were given, touch files they were never meant to (configuration, credentials, database structure), invent a business rule instead of asking whether one exists, or plow ahead when there were actually two reasonable ways to do something and only a person could say which one was right.

None of that is a reason to avoid using AI assistants — it's a reason to put a consistent set of guardrails around the work, the same way a company puts guardrails around any powerful tool its people use. OpsGate is that guardrail, applied automatically and consistently rather than relying on every individual task to remember the rules.

3. What OpsGate actually does

It defines what's off-limits

Every project has files that should never be touched by routine work: credentials, environment configuration, database migration files, deployment settings, and the project's own governance files. OpsGate maintains this “never touch” list per project and enforces it automatically — an AI assistant simply cannot get a green light to modify one of these, no matter how the request is phrased.

It requires the work to stay inside its stated scope

Before implementation starts, OpsGate checks that the request has a clear, bounded outcome and that the files it plans to touch are the ones it's actually supposed to touch. An assistant can't quietly widen the scope of a task mid-way through without that being flagged.

It knows the difference between routine work and work that needs a green light

Most day-to-day changes — fixing a bug, adjusting a screen, adding a field — are allowed to proceed on their own once they pass the scope and protected-file checks above. A smaller set of higher-stakes categories — restructuring a database, deleting something, changing permissions, touching packages or deployment configuration — require an explicit go-ahead before OpsGate will let them proceed at all, regardless of who's asking.

It pauses and asks a human, but only when it actually matters

OpsGate does not stop and ask about every little decision — that would defeat the point of using an AI assistant at all. It only pauses in three specific situations:

- **The next step genuinely isn't knowable yet** — even after looking, the assistant can't determine what to do next without someone weighing in.
- **There are two equally valid ways to do it** — and nothing in the existing rules, conventions, or evidence favors one over the other.
- **Doing the work as asked would quietly expand what was agreed to** — the assistant would have to decide, on its own, to go further than the task actually authorized.

In every other situation, the work proceeds without interruption. When one of these three cases does come up, the assistant stops completely, states exactly what it needs decided, lays out the real options and their consequences, and waits. Nothing continues until a person replies with a decision.

4. Who's involved

Role	What it does
A human (you, or someone on your team)	Sets the actual goal, approves higher-stakes work, and answers the occasional judgment call OpsGate surfaces.
Claude	Turns a plain-language request into a well-formed, self-contained task, checks it against OpsGate's rules up front, and hands a ready-to-execute version of it to Replit.
A Replit Agent	Does the actual implementation work inside your project, re-checking the same rules along the way and reporting back what it did and how it verified it.
OpsGate	The referee. It doesn't write any code itself — it defines and enforces the rules both Claude and Replit have to follow, and is the one thing both of them check against.

The reason both Claude and Replit check against the same OpsGate rules, rather than each having their own copy of them, is simple: one place to define the rules means one place to change them, and no risk of the two assistants quietly drifting out of agreement with each other over time.

5. What connecting a project to OpsGate looks like

From your side, this is a one-time setup, not an ongoing task. A project is registered with OpsGate once — it's given an identity and a credential, and told which parts of the codebase are its normal working area versus permanently off-limits. After that, every AI-assisted change to that project automatically goes through OpsGate's checks in the background. There's nothing to remember to do differently day-to-day.

If a project is ever moved, restructured, or its off-limits list needs to change, that's a quick update to its registration — not a rebuild of anything.

6. What this means day-to-day

- **Routine changes feel exactly the same as before** — a well-scoped bug fix or small feature goes through without any extra friction, because it passes every check automatically.
- **Higher-stakes changes get a deliberate pause** — schema changes, deletions, permission changes, and similar categories always require an explicit authorization, on purpose.
- **You'll occasionally be asked a real question** — not a rubber-stamp approval request, but an actual decision point the assistant genuinely cannot resolve on its own. These are rare by design, and each one comes with the specific options and their consequences already laid out.
- **Nothing about this changes what gets built** — OpsGate governs how AI assistants are allowed to work, not what your team decides to build. It has no opinion on product direction.

7. Where things stand today

OpsGate is live and actively used. Its rules, protected files, and higher-stakes categories are already enforced on real work today, not as a future plan. The system has been through several rounds of dedicated security and correctness review, including adversarial testing performed specifically to find and close gaps before they could matter.

Where it runs is the one thing that is not yet where it should be. Today OpsGate runs on a single machine as an interim arrangement — which means every connected project depends on that one machine being switched on and reachable. Moving it onto proper server infrastructure is a required next step, not an optional improvement; the packaging for that move is already built and tested, and the technical documentation sets out the recommended approach.

If you take away one thing

OpsGate exists so that using AI assistants on real projects doesn't require trusting each individual task to remember every rule — the rules are enforced automatically, the same way, every time, and a person is only pulled in when a decision genuinely needs one.